Neuro-symbolic Approaches to the SAT Problem

Intelligent Systems for Pattern Recognition
Reinforcement Learning

Donato Meoli
Università di Pisa

Survey with coding project



The SAT problem

- SAT: given a Boolean formula in conjunctive normal form $\varphi = \bigwedge_j (\bigvee_{\ell \in C_j} \ell)$, is there an assignment of the variables that satisfies all its clauses? It was the first problem proven NP-complete, and it is the engine of verification, planning and synthesis.
- Modern solvers are CDCL: they *decide* a variable, *propagate*, and on a *conflict* learn a clause and backjump.
- The branching heuristic (which variable to decide next, usually VSIDS) drives the performance, but its activity counters carry little information at the start of the search.
- This project: learn the branching heuristic with two neuro-symbolic methods, port both to a current PyTorch stack, check the port against the released checkpoints, and measure where graph attention helps.

Roadmap

Two ways to learn a SAT heuristic:

- Graph-Q-SAT: value-based reinforcement learning (DQN) with a graph network on the graph of the formula, which applies to formulas of any size;
- AlphaZeroSAT: self-play and MCTS with a convolutional policy and value network on the clause $\times$ variable matrix, of fixed size.

What I did:

- ported Graph-Q-SAT from PyTorch 1.1 and AlphaZeroSAT from TensorFlow 1.x and GSL to one current PyTorch / PyTorch Geometric stack;
- GAT-Q-SAT, which adds graph attention to Graph-Q-SAT, compared with it at matched training set and with two controls;
- the restricted heuristics of Shirokikh et al.

Plan: learning on the graph of a formula $\rightarrow$ Graph-Q-SAT and GAT-Q-SAT $\rightarrow$ AlphaZeroSAT $\rightarrow$ implementation $\rightarrow$ results.

Background: learning on the graph of a CNF

- The CNF as a graph
- Message Passing Neural Networks (MPNN)
- NeuroSAT, the first graph model for SAT

(ISPR – survey)

Gilmer et al., *Neural Message Passing for Quantum Chemistry.*
Selsam et al., *Learning a SAT Solver from Single-Bit Supervision.*

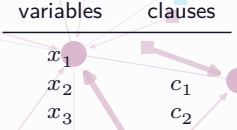
The CNF as a graph

A CNF is naturally a bipartite graph:

- a node for every variable (or literal) and every clause;
- an edge between a variable and each clause in which it occurs, with the polarity as edge feature (in the view by literals, also an edge between x and $\bar{x}$).

The representation is invariant under permutations of variables and clauses, and one network applies to formulas of any size, which the fixed-size CNN cannot do. It is the representation of NeuroSAT and of Graph-Q-SAT.

$$\varphi = (x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_2 \vee x_3)$$



the edges carry the polarity $\{+, -\}$ of the literal.

Message Passing Neural Networks (MPNN)

A general framework for learning on graphs (Gilmer et al., 2017). Each node v holds a state h_v^t, and for T rounds it exchanges messages with its neighbors $\mathcal{N}(v)$:

$$\text{message:} \quad m_v^{t+1} = \sum_{w \in \mathcal{N}(v)} M_t(h_v^t, h_w^t, e_{vw})$$

$$\text{update:} \quad h_v^{t+1} = U_t(h_v^t, m_v^{t+1})$$

$$\text{readout:} \quad \hat{y} = R(\{\, h_v^T : v \in G \,\})$$

- M_t (message), U_t (update, often a GRU or an LSTM) and R (readout) are learned and shared across the nodes and the rounds.
- Gated GNNs, GCN, GAT and the encode–process–decode network are all instances of this template.

NeuroSAT: a SAT solver from single-bit supervision

Literal and clause embeddings $L^{(t)} \in \mathbb{R}^{2n \times d}$, $C^{(t)} \in \mathbb{R}^{m \times d}$; bipartite adjacency M; perceptrons $L_{\text{msg}}, C_{\text{msg}}, L_{\text{vote}}$; LSTM updates L_u, C_u. One round:

$$(C^{(t+1)}, C_h^{(t+1)}) \leftarrow C_u([\, C_h^{(t)},\ M^\top L_{\text{msg}}(L^{(t)})\,])$$

$$(L^{(t+1)}, L_h^{(t+1)}) \leftarrow L_u([\, L_h^{(t)},\ F(L^{(t)}),\ M\, C_{\text{msg}}(C^{(t+1)})\,])$$

after T rounds, vote and average:

$$L_*^{(T)} \leftarrow L_{\text{vote}}(L^{(T)}), \qquad y^{(T)} \leftarrow \text{mean}\,(L_*^{(T)}).$$

F swaps each literal with its complement.

- Trained only as a classifier of satisfiability (one bit per formula).
- Yet it learns to find satisfying assignments, which appear in the votes of the literals.
- It established the graph view of a CNF, on which Graph-Q-SAT is built.
- Limitation: a standalone solver, not competitive in running time with CDCL.

- Reinforcement learning for branching
- Background
  - Deep Q-Learning (DQN)
  - Graph Networks
- Model: encode–process–decode
- Graph attention (GAT) and two controls

(ISPR + RL – survey with extra coding)

Kurin et al., *Can Q-Learning with Graph Networks Learn a Generalizable Branching Heuristic for a SAT Solver?*

Graph-Q-SAT

&

GAT-Q-SAT

Branching as a Markov Decision Process

Graph-Q-SAT turns the CDCL search into an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- state s: the formula simplified by the partial assignment, with the learned clauses, as a bipartite graph;
- action a: an unassigned variable with a truth value, i.e., one decision;
- reward $r = -0.1$ at every non-terminal step and 0 at termination, so that maximizing the return minimizes the number of decisions.

The goal is the action-value function

$$Q^\star(s,a) = \mathbb{E}\big[\, r + \gamma \max_{a'} Q^\star(s',a') \,\big] \qquad \text{(Bellman)},$$

and the heuristic is $\pi(s) = \arg\max_a Q^\star(s,a)$ over the variable nodes.

Deep Q-Learning (DQN)

$Q^\star$ is approximated by a network Q_θ, fitted by minimizing the temporal-difference error over a replay buffer $\mathcal{D}$:

$$\mathcal{L}(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\Big(\underbrace{r + \gamma \max_{a'} Q_{\theta^-}(s', a')}_{\text{TD target}} - Q_\theta(s,a)\Big)^2\right]$$

- the target network θ^-, a copy of θ refreshed every 10 updates, stabilizes the target;
- the replay buffer (20000 transitions, sampled uniformly) breaks the correlation between consecutive transitions;
- ε-greedy exploration: a random action with probability ε, otherwise $\arg\max_a Q_\theta$.

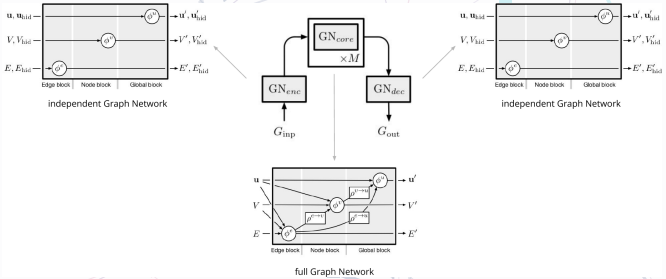
The number of actions changes along an episode (two per unassigned variable), and hence Q_θ is a graph network that scores each variable node.

Model: the encode--process--decode graph network

A graph-network block (Battaglia et al.) updates the edges, then the nodes, then the global attribute:

$$e'_{ij} = \phi^e(e_{ij}, v_i, v_j, u)$$
$$v'_i = \phi^v(\sum_{j \in \mathcal{N}_i} e'_{ij}, v_i, u)$$
$$u' = \phi^u(\bar{e}', \bar{v}', u)$$

ϕ are perceptrons; the sum is the aggregation ρ of the figure. The encoder and the decoder act on each element independently; the core block is applied $M = 4$ times with shared weights, and the decoder gives two Q-values per variable.



encode–process–decode: $\mathrm{GN}_{enc} \to \mathrm{GN}_{core} \times M \to \mathrm{GN}_{dec}$.

Configuration and measure

The configuration of Kurin et al.:

- node features: 2-dimensional one-hot (variable or clause); edge features: the polarity; the global attribute: a constant;
- embeddings of size 32 from the encoder, perceptrons with 64 hidden units in the core, $M = 4$ rounds; 71144 parameters;
- Adam at $2 \cdot 10^{-5}$, $\gamma = 0.99$, minibatches of 64, 50000 updates, at most 500 decisions of the model per episode (the cap).

The measure, MRIR:

$$\text{MRIR} = \text{median}\, \frac{\#\text{iter}_{\text{MiniSat}}}{\#\text{iter}_{\text{model}}}$$

the Median Relative Iteration Reduction over the instances of a family: above 1 the model takes fewer decisions than MiniSat. After the cap, VSIDS takes the remaining decisions.

Improvement idea: graph attention

- In the core block a node sums the messages of all its neighbors with the same weight; on a random formula the clauses are statistically alike, but in a structured one they often play different roles (in graph coloring, "at least one color" for a node and "different colors" for the ends of an edge).
- Idea: weight the neighbors by learned coefficients that depend on the edge embeddings, i.e., a Graph Attention Network (GAT) inside the core block.
- In the terms of Battaglia et al., a non-local block inserted in the full graph-network block.

Veličković et al., *Graph Attention Networks* (ICLR 2018).

Graph Attention Networks (GAT)

For a node i with neighbors $\mathcal{N}_i$ and a shared linear map W, the attention weights each message by a learned relevance:

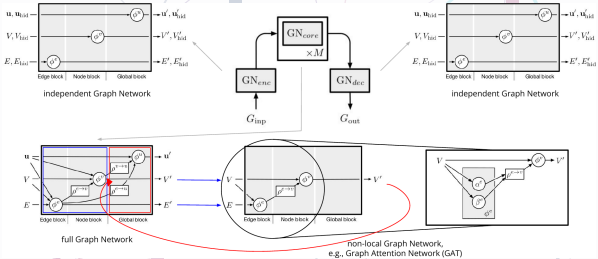
$$\begin{aligned} \text{score:} \quad & \omega_{ij} = \mathsf{LeakyReLU}(\mathbf{w}^\top [\, W v_i \,\|\, W v_j \,]) \\ \text{normalize:} \quad & \alpha_{ij} = \frac{\exp \omega_{ij}}{\sum_{j' \in \mathcal{N}_i} \exp \omega_{ij'}} \\ \text{aggregate:} \quad & v_i^+ = \sum_{j \in \mathcal{N}_i} \alpha_{ij} W v_j \end{aligned}$$

- multi-head: K independent attentions, concatenated or averaged;
- edge-featured: the score also reads the embedding of the edge, which carries the polarity of the literal.

GAT-Q-SAT and the two controls

GAT-Q-SAT adds two edge-featured attention layers (3 heads, then 1) between the node and the global update of the core block, and keeps the sum: 113256 parameters, about 60% more. A gain may then be due to the attention or to the capacity, and two controls separate them:

- Graph-Q-SAT wide (104 hidden units, 113504 parameters, no attention);
- attention aggregation (the sum replaced by an attention-weighted mean, 71627 parameters).



the core block with a non-local block, a GAT with edge attention α^e and messages β^e.

- Search-based reinforcement learning for branching
- Background
  - Monte Carlo Tree Search (MCTS)
  - AlphaZero self-play
- Model: a convolutional policy and value network
- Results

(ISPR + RL – survey with coding)

Wang & Rompf, *From Gameplay to Symbolic Reasoning: Learning SAT Solver Heuristics in the Style of Alpha(Go) Zero.*

AlphaZeroSAT

Monte Carlo Tree Search (MCTS)

Solving is treated as a game: a node is a partial assignment, a move a branching literal. From the root, four steps are repeated:

1. select the child that maximizes the PUCT score

$$a_t = \arg\max_a \Big(Q(s,a) + \underbrace{c_{\text{puct}} P(s,a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}}_{U(s,a)} \Big);$$

2. expand a leaf and evaluate it with the network $(P, v) = f_\theta(s)$;
3. bootstrap from the value v;
4. back up: $N \mathrel{+}= 1$, $W \mathrel{+}= v$, $Q = W/N$.

The visit counts give an improved policy, $\pi(a \mid s) \propto N(s,a)^{1/\tau}$.

AlphaZero: self-play policy iteration

A single network $(p, v) = f_\theta(s)$ gives a policy p over the moves and a value $v \in [-1, 1]$ (how likely the search is to succeed from s).

- Self-play: MCTS, guided by f_θ, gives a stronger policy π, played to the end to obtain an outcome z.
- Learning: v is regressed on z and p on π; MCTS improves the policy and the network evaluates it:

$$\ell = \underbrace{(z - v)^2}_{value} \quad \underbrace{- \pi^\top \log p}_{policy(cross\text{-}entropy)} \quad + \quad \underbrace{c\|\theta\|^2}_{regularization} \quad .$$

- No human data and no hand-made heuristic: the heuristic is learned from self-play alone.

AlphaZeroSAT: the model

State: the CNF as a fixed-size clauses $\times$ variables matrix (entries in $\{+1, 0, -1\}$ by polarity, as image channels).

Network: a convolutional trunk ($32@8\times8$, $64@4\times4$, $64@3\times3$), then two heads:

$$policy\ p = \text{softmax}(\text{logits} + (\text{mask} - 1)\cdot\infty) \in \Delta^{2n}$$

$$value\ v = \tanh(\cdot) \in [-1, 1]$$

the literals of the variables absent from the remaining clauses are masked.

Model1/2/3 differ in the heads (fully connected or convolutional), as in the original design.

Environment: MiniSat extended into an MCTS game that supports hypothetical simulations.

Measure: the mean number of branching decisions to solve (lower is better), as in the original paper.

Result (`uf20-91`, 25 cycles of self-play): 6.70 mean decisions on 64 test problems; the evaluation after each cycle, on 16 problems, ranges from 3.4 to 9.4, too noisy to show a trend.

Each decision is an MCTS search, much slower than a decision of VSIDS.

Limitation: with a fixed-size matrix the network cannot be applied to other sizes, which is the motivation of the graph methods.

- The port
- Restricted heuristics
- A test of the port
- Where attention helps

Implementation & Results

The port: one current PyTorch stack

- Graph-Q-SAT was written for PyTorch 1.1 with pinned compiled extensions (`torch-scatter`, `torch-sparse`); AlphaZeroSAT for TensorFlow 1.x, with GSL in its C++ environment. Neither can be installed today.
- Graph-Q-SAT: the aggregations from `torch_geometric.utils.scatter`, the environment built without `gym.make`, the wrapper of MiniSat rebuilt against `numpy` 2, the checkpoints of 2021 loaded with the current `GATConv`.
- AlphaZeroSAT: the network and the AlphaZero loss rewritten in PyTorch, GSL removed (the Dirichlet noise from C++11 `<random>`); it trains end to end on a GPU.
- Both run on Colab with the checkpoints on Drive; the instances of Graph-Q-SAT live in its repository, those of AlphaZeroSAT in `data/`.

Restricted heuristics: fewer forward passes

Graph-Q-SAT reduces the iterations, but a forward pass of the network at each decision costs more than VSIDS. Three ways of querying the network less (Shirokikh et al., 2023), at evaluation time:

- early release (`--release_after T`): the model takes the first T decisions, VSIDS the others;
- action pool (`--action_pool_size L`): one forward pass gives the L best actions, executed in order;
- warm start (`--warmstart_release`): the Q-values of the root initialize the activities of VSIDS, $\text{activity}(x) = -1/\max\left(Q(x, \mathsf{T}), Q(x, \mathsf{F})\right)$, which then runs unaided.

On the instances of this project the search is too short for the saving to show, and they are not measured.

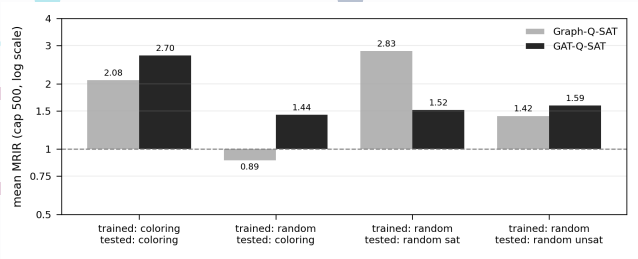
A test of the port

- The checkpoints trained in 2021 on the original stack, evaluated again on the new one, and compared with the logs of 2021 instance by instance.
- In 63 of 86 evaluations the iterations of the model coincide on every instance; in the other 23 the median changes by at most 0.03:

`flat30-60`, cap 500	2021	2026
Graph-Q-SAT	1.833	1.833
GAT-Q-SAT	2.000	2.000

- The training path was checked by retraining the two coloring models from scratch on the new stack.

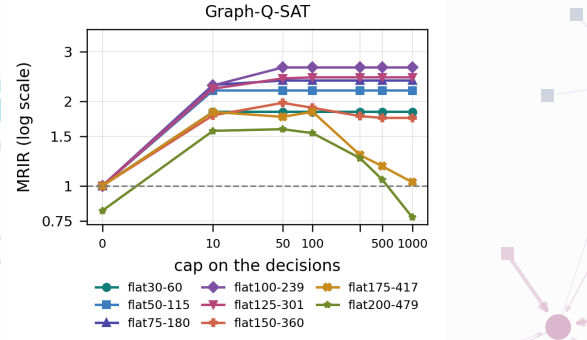
Results: Graph-Q-SAT and GAT-Q-SAT at matched training set

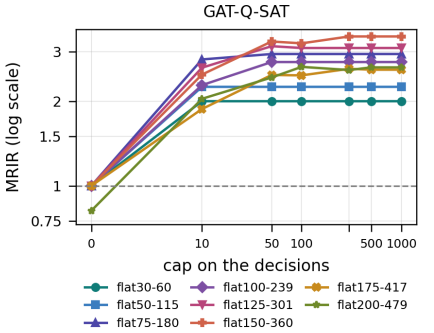


Mean MRIR at cap 500; light gray Graph-Q-SAT, dark gray GAT-Q-SAT.

- Graph coloring: GAT-Q-SAT ahead, 2.70 against 2.08.
- Trained on random formulas, tested on coloring: Graph-Q-SAT below MiniSat (0.89), GAT-Q-SAT above it (1.44).
- Satisfiable random 3-SAT: Graph-Q-SAT ahead (2.83 against 1.52).

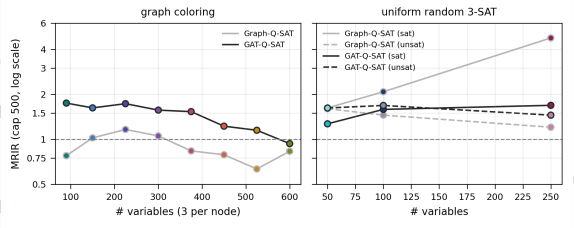
Results: the MRIR against the cap on graph coloring





One line per family, each in its color (the same in every figure); on the large families the decisions of Graph-Q-SAT after the first few tens make the search longer.

Results: the gap grows with the size



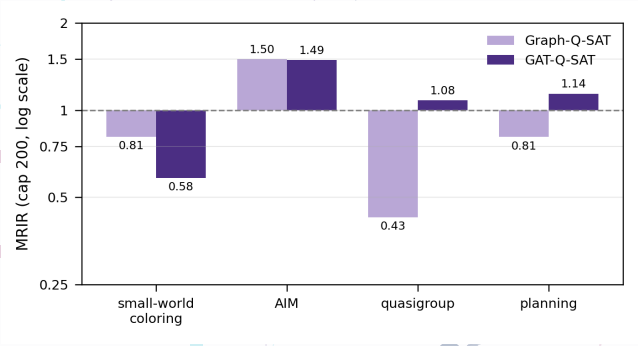
Trained on random formulas: MRIR against the size (cap 500).

Trained and tested on graph coloring, MRIR at cap 500:

family	Graph	GAT	Δ
flat30-60	1.83	2.00	+0.17
flat125-301	2.43	3.09	+0.66
flat150-360	1.74	3.40	+1.66
flat200-479	1.05	2.64	+1.59

Graph-Q-SAT degrades on the large instances while GAT-Q-SAT degrades much less.

Results: a change of domain, with no retraining



The coloring-trained models on the thirteen structured SATLIB families up to 2000 variables and 150000 clauses; cap 200, MRIR against the stronger MiniSat.

[On how many families each model is above MiniSat and above the other.]

Results: fewer iterations, and the running time

	cap 50		cap 500		cap 1000	
	sec	MRIR	sec	MRIR	sec	MRIR
Graph-Q-SAT	2.55	1.59	26.51	1.05	57.67	0.77
GAT-Q-SAT	3.52	2.43	12.20	2.64	12.19	2.64

flat200-479, median seconds per instance

Each decision of a model is a forward pass of the network: GAT-Q-SAT costs more per decision, but at larger caps it is the faster of the two, since it terminates in fewer decisions. Both take seconds where MiniSat takes a fraction of a second, which is the point of the restricted heuristics.

Where the field went, and conclusions

- NeuroSAT → NeuroCore → NeuroBack (ICLR 2024): a graph transformer queried once before the search, whose inference is paid offline; the same direction of attention as GAT-Q-SAT.
- Graph attention helps on structured problems (graph coloring), with a gap that grows with the size, and also when the model is trained on random formulas; on satisfiable uniform random 3-SAT Graph-Q-SAT is better.
- [The controls and the transfer panel, from the note.]
- The fixed-size network of AlphaZeroSAT cannot be applied to other sizes, which is the motivation of the graph methods.

Code and slides: `github.com/dmeoli/NeuroSAT`; the note on GAT-Q-SAT: `github.com/dmeoli/GQSAT`.

Thank you for your *attention*!

github.com/dmeoli/NeuroSAT